

Examen SLE

Tous documents autorisés, calculatrice autorisée, ordinateurs et autres portables/mobiles
interdits

Rendre les deux parties sur des feuilles distinctes

1 Hiérarchie mémoire et détection de visage (12 pts)

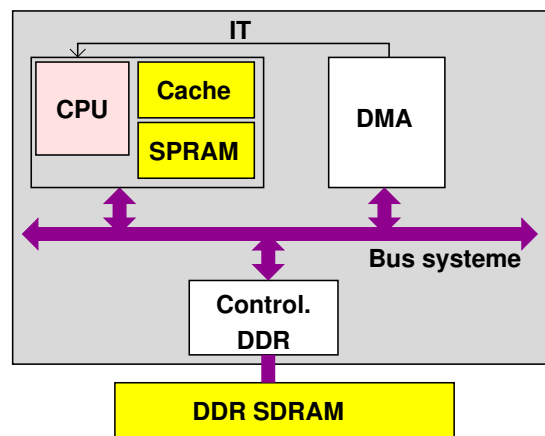


FIGURE 1 – Architecture mémoire

On souhaite réaliser un algorithme de détection de visage à l'aide d'un processeur 32 bits qui dispose d'un cache et d'une Scratch-Pad RAM (SPRAM) ainsi qu'illustré sur la figure 1. Le processeur accède au cache et à la SPRAM en un seul cycle d'horloge. Le processeur est connecté à la mémoire externe à travers un bus système 32 bits. Un DMA est présent dans le système afin de réaliser différents transferts entre les mémoires. Le DMA est configuré par l'adresse de départ, l'adresse d'arrivée et la quantité de données transférées. Il émet une interruption lorsque le transfert est terminé. Il contient une mémoire de la taille du burst maximum.

1.1 Fonctionnement du DMA (2 points)

▷ **Question 1 :**

Rappeler en deux phrases la différence entre une SPRAM et un cache.

On appelle l_m la latence d'accès à la mémoire externe et l_{SPRAM} la latence d'accès à la SPRAM depuis le DMA. b_{max} est la taille maximum d'un burst sur le bus système (exprimé en octets). Un burst est constitué d'une séquence de mots de 32 bits.

▷ **Question 2 :**

Dessiner le chronogramme d'une séquence de transfert DMA d'un bloc de données de taille b_{max} provenant de la mémoire externe et écrites en SPRAM.

t_{IT} est le temps minimum de réponse du processeur à une interruption. Cela correspond au nombre de cycles minimum entre le moment où une interruption apparaît et l'exécution de la première instruction de la routine de traitement d'interruption. Ce temps inclus la sauvegarde de contexte, etc...

▷ **Question 3 :**

Comment synchroniser un DMA avec le logiciel en utilisant des interruptions ? Dessiner un chronogramme type pour le transfert d'un paquet de n KOctets.

▷ **Question 4 :**

Exprimer le nombre de cycles minimum pour transférer entre la mémoire externe et la SPRAM un paquet de données de n KOctets en fonction de n , l_{SPRAM} , l_m , b_{max} et t_{IT}

1.2 Application (10 points)

On souhaite réaliser un algorithme de détection de visage dans une image stockée en mémoire externe. Un pixel de l'image est codé sur 8 bits. L'image d'entrée I est de taille $t_x \times t_y$.

Les pixels sont stockés en mémoire dans l'ordre canonique. C'est-à-dire que l'adresse d'un pixel se calcule par :

$$\text{adresse}(I(x, y)) = x + y * t_x$$

Avec x variant de 0 à $t_x - 1$ et y variant de 0 à $t_y - 1$. Le coin 0,0 est en haut à gauche et le coin $t_x - 1, t_y - 1$ est en bas à droite.

Valeurs numériques

Dans la suite :

- $t_x = t_y = 512$
- $l_m = 20$ cycles
- $l_{SPRAM} = 10$ cycles
- $b_{max} = 16$ Octets
- $t_{IT} = 400$ cycles
- La SPRAM fait 4 KOctet

L'algorithme de détection de visage commence par calculer l'image intégrale de l'image de départ. Un pixel de l'image intégrale $II(x, y)$ prend pour valeur la somme de tous les pixels du rectangle compris entre l'origine (en haut à gauche) et le pixel $I(x, y)$ de l'image de départ.

Le calcul de l'image intégrale se fait en deux passes : une intégrale verticale suivie d'une intégrale horizontale. On exécute l'algorithme suivant :

```
/* 1ere passe : intégrale verticale */
for(=0; x<tx; x++)                /* pour chaque colonne */
{
    II(x,0)=I(x, 0);                /* initialise l'intégrale */
    for(y=1; y<ty; y++)            /* pour chaque ligne */
        II(x,y) =II(x,y-1)+I(x,y);

}

/* 2nde passe : intégrale horizontale */
for(y=0; y<ty; y++)                /* pour chaque ligne */
    for(x=1; x<tx; x++)            /* pour chaque colonne */
        II(x,y) =II(x-1,y)+I(x,y);
```

Une zone $l \times h$ de l'image I a une largeur de l pixels et une hauteur de h pixels.

▷ **Question 5 :**

Combien de temps minimum faut-il pour charger une ligne de $t_x \times 1$ pixels dans la SPRAM ?

▷ **Question 6 :**

Combien de temps minimum faut-il pour charger une colonne de $1 \times t_y$ pixels dans la SPRAM ?

Indication : Les pixels d'une colonne ne sont pas à des adresses contiguës

▷ **Question 7 :**

Combien de lignes et colonnes l'algorithme précédent nécessite-t-il de charger ?

- Dans la passe verticale
- Dans la passe horizontale

En déduire le temps total passé à charger les données pour le calcul de l'image intégrale.

Nous allons essayer d'optimiser cet algorithme et améliorer le temps de chargement.

Tout d'abord nous remarquons qu'il est plus intéressant de charger plusieurs colonnes simultanément car un mot mémoire de 32 bits contient 4 pixels consécutifs, qui appartiennent à 4 colonnes.

▷ **Question 8 :**

Combien de temps faut-il pour charger une bande verticale de $l \times t_y$ pixels dans la SPRAM ?

Donner l'expression analytique puis le temps :

- pour $l=2$
- pour $l=4$
- pour $l=6$
- pour $l=8$

Nous remarquons aussi que deux lignes de l'image sont à des adresses consécutives.

▷ **Question 9 :**

Combien de temps faut-il pour charger une bande horizontale de $t_x \times h$ pixels dans la SPRAM ?

Donner l'expression analytique puis le temps :

- pour $h=2$
- pour $h=4$
- pour $h=6$
- pour $h=8$

La SPRAM faisant 4 KOctets, on peut au maximum stocker 8 lignes ou 8 colonnes en mémoire.

▷ **Question 10 :**

A partir de l'algorithme original, écrire un algorithme qui charge les zones de données en SPRAM, et fait les calculs sur plusieurs lignes ou colonnes. On utilisera une fonction `charge_zone(x, y, l, h)` qui charge en SPRAM une zone de coin supérieur gauche de coordonnée x, y et de taille $l \times h$ (il n'est pas demandé de détailler cette fonction, juste de l'utiliser dans le nouvel algorithme).

▷ **Question 11 :**

En déduire le temps total passé à charger les données pour le calcul de l'image intégrale.

Une optimisation supplémentaire consisterait à charger un bloc de taille $B_x \times B_y$ en SPRAM, puis faire les deux passes horizontale et verticale sur tout le bloc. Le problème est que le calcul d'un bloc dépend du bloc voisin. Sur la figure suivante nous voyons que le calcul du bloc B nécessite le bord droite du bloc A, et le calcul du bloc F nécessite le bord inférieur du bloc B et le bord droite du bloc E.

A	B	C	D
E	F	G	H
I	J	K	L
M	N	O	P

Une solution consiste à calculer les blocs horizontalement, ligne après ligne, en utilisant les valeurs des blocs précédemment calculés. On a alors l'algorithme suivant :

```
for(by=0; by<ty; by+=By) /* pour chaque ligne de bloc */
{
    for(bx=0; bx<tx; bx+=Bx) /* pour chaque bloc de la ligne*/
    {
        /* 1ere passe : intégrale verticale */
        for(x=0; x<Bx; x++)          /* pour chaque colonne du bloc */
        {
            /* initialise l'intégrale sur la colonne*/
            if (by!=0)                /* si ce n'est pas le premier bloc,*/
                II(bx+x,by)=II(bx+x,by-1)+I(bx+x,by); /* utilise le calcul précédent */
            else
                II(bx+x,0)=I(bx+x,0);

            for(y=1; y<By; y++)        /* pour chaque ligne du bloc */
                II(bx+x,by+y) =II(bx+x,by+y-1)+I(bx+x,by+y);

        }
        /* 2nde passe : intégrale horizontale */
        for(y=0; y<By; y++)            /* pour chaque ligne du bloc */
            for(x=0; x<Bx; x++)        /* pour chaque colonne du bloc */
                if (bx+x!=0)
                    II(bx+x,by+y) =II(bx+x-1,by+y)+I(bx+x,by+y);
    }
}
```

▷ **Question 12 :**

Quel est le temps nécessaire pour charger un bloc de taille $B_x \times B_y$ en SPRAM ?

Attention : le chargement de deux lignes de B_x pixels ne peut pas se faire avec un seul transfert DMA lorsque les données ne sont pas à des adresses consécutives ! A l'inverse, si les données sont consécutives en mémoire, un seul transfert DMA suffit.

Donner les équations pour chacun des cas.

▷ Question 13 :

En déduire le temps total passé à charger les données pour le calcul de l'image intégrale dans les cas suivant :

- $B_x = 32, B_y = 32$ (1 bloc=1 KOctets)
- $B_x = 64, B_y = 32$ (1 bloc=2 KOctets)
- $B_x = 128, B_y = 32$ (1 bloc=4 KOctets)
- $B_x = 512, B_y = 8$ (1 bloc=4 KOctets)

Rendre les deux parties sur des feuilles distinctes

2 Déroulement de boucle et ordonnancement d'instructions (8 points)

Le produit scalaire de 2 vecteurs $\vec{x} = (x_1, \dots, x_n)$ et $\vec{y} = (y_1, \dots, y_n)$ dans un espace de dimension n se calcule comme $\sum_{i=1}^n x_i \cdot y_i$. Traduit en assembleur MIPS, cela donne (avec F2 initialement à zéro et R1 pointant sur x_1 et R2 sur y_1 , sachant que les $\vec{x}$ et $\vec{y}$ sont en fait des tableaux de flottant double précision - 8 octets -, et R3 contient initialement n) :

```
loop: L.D F0, 0(R1)    % charge x[i]
      L.D F4, 0(R2)    % charge y[i]
      MUL.D F0, F0, F4 % calcul de x[i] * y[i]
      ADD.D F2, F2, F0 % somme à l'existant
      ADDUI R1, R1, 8  % passe à l'x suivant
      ADDUI R2, R2, 8  % passe à l'y suivant
      ADDUI R3, R3, -1 % décrémente le compteur
      BNEZ R3, loop   % itère si pas au bout
      S.D F2          % sauve le resultat
```

On a les gels (*stall*) suivants (identiques à ceux pris en cours) :

Instruction produisante	Instruction consommante	Gels (cycles)
FP ALU op	FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
FP op	INT ALU op	0
INT ALU op	INT ALU op	0
INT ALU op	branch	1

▷ Question 14 :

L'instruction qui suit le branchement ne fait pas partie de la boucle. Ré-écrivez la boucle telle quelle en faisant apparaître explicitement les cycles de gels dues aux dépendances et anti-dépendances de données.

▷ Question 15 :

Réordonnez les instructions de sorte à minimiser ce nombre de cycles. Donnez le CPI de la boucle.

▷ Question 16 :

S'il reste des cycles de stall, déroulez la boucle du nombre de cycles nécessaires à les faire disparaître. Donnez le CPI de la boucle.

▷ Question 17 :

Appliquez la technique du pipeline logiciel à la boucle originale pour en faire disparaître les dépendances.

3 Réseau sur puce (8 points)

Cet exercice porte sur le réseau dont la topologie, appelée Spidergone, est donnée à la figure 2. Les hypothèses sont les suivantes : un paquet qui va d'un routeur à l'un de ses voisins (directement connecté par un lien) met 1 cycle ; les connexions entre les nœuds (liens) sont des canaux bidirectionnels, chaque canal possédant une bande passante "locale" notée b .

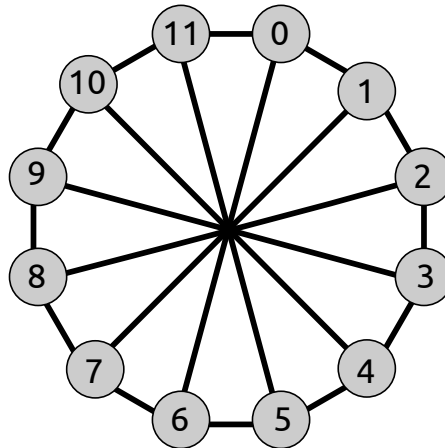


FIGURE 2 – Réseau de type Spidergone avec $N = 12$ nœuds

▷ **Question 18 :**

Quelle contrainte existe-t-il sur le nombre de nœuds N du réseau ;

▷ **Question 19 :**

Déterminez le nombre de liens L pour $N = 12$ et en déduire la bande passante totale du réseau. Donnez ensuite une expression analytique de L en fonction de N ;

▷ **Question 20 :**

Quelle est la bande passante (*bisection bandwidth* qui passe sur les canaux traversés lorsque l'on coupe le réseau à 12 nœuds en *deux parties égales* ? Donnez en l'expression analytique en fonction de N ; Donnez le diamètre (le nombre de nœuds *hop count* qu'il faut traverser pour qu'un paquet voyage entre les nœuds les plus distants, incluant la source *ou* la destination), toujours sur cet exemple à 12 nœuds. Donnez un exemple de tel chemin (genre $xx \rightsquigarrow yy$, ou xx et yy sont les numéros des nœuds). Exprimez le diamètre analytiquement ;

▷ **Question 21 :**

Donnez le nombre moyens de nœuds à traverser (distance moyenne) pour un paquet du réseau en supposant une équiprobabilité de répartition, d'abord pour le réseau à 12 nœuds puis en fonction de N . Par ex., on peut supposer que le nœud 0 envoie un paquet à chacun des autres nœuds du réseau.